

PsyberShield Manual

A combined manual covering usage, architecture notes, and project history.

User Guide

Source: `'docs/turan-user-guide.md'`

PsyberShield User Guide

PsyberShield is a Python-based security visibility and response tool for small servers and web applications.

This guide is a living document. It describes the current CLI behavior, the command set, the important flags, and what to expect from each workflow.

The preferred CLI command is `'pshield'`. `'psybershield'` and `'turan'` remain compatibility aliases during the transition.

For version history, see `[docs/changelog.md](docs/changelog.md)`.

This guide and the changelog should be updated together when PsyberShield's user-facing behavior changes.

What PsyberShield Does

PsyberShield helps you:

- scan a target URL
- crawl in-scope pages
- discover a local app target on a VPS when no URL is supplied
- inspect local environment and server layout
- generate safe fix artifacts
- apply one real local fix lane with backup and validation
- save reports, baselines, audit history, and comparison summaries

PsyberShield is intentionally defensive:

- it warns before risky commands
- it never prints secret values
- it prefers read-only behavior unless you explicitly ask for fixes
- it keeps a clear trail in the terminal and in saved outputs

Command Map

Command	What it does	Typical use
---	---	---
'scan'	Scans one target page or a discovered local target; supports browser-assisted login	
'crawl'	Crawls in-scope links across multiple pages; supports browser-assisted login and	
'report'	Re-renders or previews a saved report	Review an existing JSON, Markdown, or HTML
'audit'	Shows the append-only audit trail	Review scans and fix events
'baseline'	Saves a scan snapshot for later comparison	Track a known-good state
'compare'	Compares two saved scans and crawl coverage	See what changed
'drift'	Compares saved reports and highlights baseline drift	Watch for scan, file, log,
'secrets'	Scans files for obvious secret exposure	Check logs and configs for leaked values
'vuln scan'	Inventories local software versions and matches a small bundled offline advisor	
'bundle'	Packages a report and related artifacts into a ZIP archive	Share a handoff bundle
'doctor'	Checks the local machine and app environment	Health check without a URL
'server-check'	Discovers the server layout and scans the local app target	VPS/server discovery
'incident'	Detects suspicious activity in logs and can apply a denylist or fail2ban snippets	
'watch'	Monitors logs, file drift, and process activity in a snapshot or follow loop; writes	
'timeline'	Shows the chronological order of findings and containment actions	Saved incidents
'integrity'	Compares monitored files against a saved baseline	File integrity drift monitoring
'fix --local'	Applies the first real local edit lane	Backup, edit, validate, rollback if needed
'demo'	Runs the local demo site	Test target for development
'demo-site'	Compatibility alias for the local demo site	Legacy command support

Quick Start

Scan a target:

```
.\env\Scripts\python.exe -m app.main scan https://example.com
```

Discover a VPS target without typing a URL:

```
.\env\Scripts\python.exe -m app.main scan
```

Crawl more than one page:

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --max-pages 20 --max-depth 2
.\env\Scripts\python.exe -m app.main crawl https://example.com --profile safe-vps
```

Generate an HTML report with a default filename under 'outputs/':

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --html-output
```

Run the server discovery workflow:

```
.\venv\Scripts\python.exe -m app.main server-check --yes
```

Check logs for suspicious activity and optionally apply containment:

```
.\venv\Scripts\python.exe -m app.main incident --logs outputs\access.log --apply-blocks
.\venv\Scripts\python.exe -m app.main integrity . --baseline baselines\integrity.json
.\venv\Scripts\python.exe -m app.main timeline outputs\incident.json --audit-log outputs\audit
.\venv\Scripts\python.exe -m app.main watch --logs outputs\access.log --json-output
.\venv\Scripts\python.exe -m app.main watch --follow --interval 30 --tail-file outputs\access.
.\venv\Scripts\python.exe -m app.main vuln scan --html-output
```

Capture fresh snapshots from live sources:

```
.\venv\Scripts\python.exe -m app.main incident --live --tail-file outputs\access.log
.\venv\Scripts\python.exe -m app.main incident --live --event-log-name System --fail2ban-output
```

Write a fail2ban-style snippet:

```
.\venv\Scripts\python.exe -m app.main incident --logs outputs\access.log --fail2ban-output out
```

View a saved incident timeline:

```
.\venv\Scripts\python.exe -m app.main timeline outputs\incident.json --audit-log outputs\audit
```

Generate rate-limit and maintenance-mode presets:

```
.\venv\Scripts\python.exe -m app.main incident --logs outputs\access.log --rate-limit-output o
```

Compare saved reports for drift:

```
.\venv\Scripts\python.exe -m app.main drift baselines\scan.json outputs\scan.json --json-output
```

Scan for obvious secret exposure:

```
.\venv\Scripts\python.exe -m app.main secrets . --markdown-output outputs\secrets.md
```

Inventory local software versions and match bundled offline advisories:

```
.\venv\Scripts\python.exe -m app.main vuln scan
.\venv\Scripts\python.exe -m app.main vuln scan --json-output outputs\vuln.json --html-output
```

```
.\venv\Scripts\python.exe -m app.main vuln scan --inventory-only
```

Bundle a report and related artifacts:

```
.\venv\Scripts\python.exe -m app.main bundle outputs\incident.json --artifact outputs\incident
```

Apply the first real local fix lane:

```
.\venv\Scripts\python.exe -m app.main fix --local --yes
```

Safety and Permission Prompt

Risky commands ask for confirmation by default:

- 'scan'
- 'crawl'
- 'server-check'
- 'incident'
- 'fix --local'

The prompt is a reminder that you should only test systems you own or have explicit permission to test.

Use '--yes' to skip the prompt in trusted automation.

Read-only commands stay quiet:

- 'doctor'
- 'report'
- 'audit'
- 'compare'
- 'baseline'

Target Resolution

'scan' and 'crawl' accept a target URL directly.

If you omit the URL, PsyberShield resolves a target in this order:

1. explicit URL on the command line
2. '--env-file'
3. the apps own '.env'

4. OS environment variables
5. server discovery on a VPS

The common environment variables are:

- 'APP_URL'
- 'TARGET_URL'
- 'BASE_URL'

When PsyberShield discovers a local app target, it prints a short 'Discovery:' line and then a fuller application context block.

Output Files and Paths

PsyberShield writes user-facing artifacts into 'outputs/'.

Supported output flags:

- '--json-output'
- '--markdown-output'
- '--html-output'
- '--output'
- '--audit-log'
- '--log-file'

You can use them in three ways:

- pass a full path
- pass a relative path
- pass the flag without a path and let PsyberShield create a timestamped filename under 'outputs/'

Examples:

```
.\env\Scripts\python.exe -m app.main scan https://example.com --html-output
.\env\Scripts\python.exe -m app.main crawl https://example.com --json-output outputs\crawl.js
.\env\Scripts\python.exe -m app.main baseline https://example.com --output
```

If you use a Windows-rooted path like 'outputs\crawl.html', PsyberShield treats it as project-relative and prints the resolved path.

'scan'

'scan' checks a single target endpoint.

Syntax:

```
.\venv\Scripts\python.exe -m app.main scan [URL]
```

Important flags:

- '--env-file PATH' reads a specific '.env'
- '--timeout SECONDS' changes the request timeout
- '--profile quick', '--profile full', and '--profile safe-vps' tune the scan defaults
- '--policy PATH' loads a policy file
- '--yes' skips the permission prompt
- '--preview-fixes' shows proposed fixes only
- '--interactive' lets you choose generate artifacts or local fixes
- '--generate-fixes' generates safe local remediation artifacts
- '--apply-fixes' is a legacy alias for '--generate-fixes'
- '--login-url', '--auth-method', '--username', '--password', '--password-env', '--cookie', '--session-file', '--storage-state', and browser auth flags enable authenticated scanning
- '--json-output', '--markdown-output', '--html-output' save reports

What to expect:

- one target page
- headers, cookies, TLS summary, exposed files, WAF signals
- suggested fix plans when report-only findings exist
- each fix plan gets a confidence label: report only, generate artifact, safe local fix, or needs manual approval
- optional saved reports in JSON, Markdown, and HTML

Example:

```
.\venv\Scripts\python.exe -m app.main scan https://example.com --json-output --markdown-output
.\venv\Scripts\python.exe -m app.main scan https://example.com --profile quick
```

'crawl'

'crawl' starts from one page and follows in-scope links across multiple pages.

Syntax:

```
.\env\Scripts\python.exe -m app.main crawl [URL]
```

Important flags:

- '--max-pages N'
- '--max-depth N'
- '--include REGEX'
- '--exclude REGEX'
- '--same-host-only' and '--allow-offsite'
- '--seed-robots' and '--seed-sitemap'
- '--env-file PATH'
- '--yes'
- authentication and browser-auth flags from 'scan'
- report output flags

What to expect:

- a crawl summary with pages visited, unique URLs, and seed sources
- findings grouped by page in the terminal and saved reports
- repeated findings collapsed into an 'Affected URLs' list
- optional seeded discovery from 'robots.txt' and 'sitemap.xml'

Example:

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --max-pages 20 --max-depth 2 -
```

Authenticated Scanning and Crawling

PsyberShield supports generic authenticated flows for protected content.

Supported auth modes:

- HTTP JSON login
- HTTP form login
- raw cookie reuse
- saved session files
- browser storage-state reuse
- browser-assisted login for JS-heavy flows

Common flags:

- '--login-url'
- '--auth-method json|form|browser'
- '--username'
- '--password'
- '--password-env'
- '--user-field'
- '--pass-field'
- '--cookie'
- '--session-file'
- '--save-session'
- '--storage-state'
- '--save-storage-state'
- '--browser-username-selector'
- '--browser-password-selector'
- '--browser-submit-selector'
- '--browser-headless' and '--browser-headed'
- '--auth-check-url'

Recommended pattern:

```
.\env\Scripts\python.exe -m app.main crawl '
https://example.com '
--login-url /auth/login '
--auth-method json '
--username alice '
--password-env PsyberShield_PASSWORD '
--auth-check-url /account
```

Browser-assisted quick example:

Browser auth requires either '--login-url' or '--storage-state' so PsyberShield can establish the session before it scans.

```
.\env\Scripts\python.exe -m app.main scan '
https://example.com '
--auth-method browser '
--login-url /auth/login '
--browser-username-selector 'input[name="identifier"]' '
--browser-password-selector 'input[name="password"]' '
--username alice '
--password-env PsyberShield_PASSWORD '
--auth-check-url /account
```

Storage-state reuse example:

```
.\env\Scripts\python.exe -m app.main crawl '
https://example.com '
--storage-state browser\storage_state.json '
--auth-check-url /account
```

What to expect:

- PsyberShield logs in before crawling or scanning protected pages
- it confirms that login worked by checking a protected URL when you provide '--auth-check-url'
- it reuses the same session for the scan or crawl

'report'

'report' re-renders or previews a saved scan report from disk.

Syntax:

```
.\env\Scripts\python.exe -m app.main report <REPORT_FILE>
```

Accepted inputs:

- '.json'
- '.md'
- '.html'
- '.htm'

Optional output flags:

- '--json-output'
- '--markdown-output'
- '--html-output'

What to expect:

- JSON reports are rendered back into the normal terminal report view
- Markdown and HTML files can be copied to new destinations
- bare output flags create files under 'outputs/'

'audit'

'audit' shows the append-only audit history.

Important flags:

- '--audit-log' and legacy alias '--log-file'
- '--last N'
- '--event NAME'
- '--target TEXT'
- '--json-output'

What to expect:

- filtered audit events in the terminal
- optional JSON export of the filtered audit list
- audit entries for scans, baselines, and fix actions

Example:

```
.\venv\Scripts\python.exe -m app.main audit --last 25 --json-output
```

'baseline'

'baseline' saves a scan snapshot for later comparison.

Important flags:

- '--label NAME'
- '--output'
- '--audit-log'
- '--timeout'
- '--policy'

What to expect:

- a baseline JSON snapshot
- a companion '.meta.json' metadata file
- discovery details saved alongside the snapshot

Example:

```
.\venv\Scripts\python.exe -m app.main baseline https://example.com --label vps-west --output
```

‘compare‘

‘compare‘ compares two saved scan files.

Important flags:

- ‘--markdown-output‘
- ‘--html-output‘

What to expect:

- fixed findings
- new findings
- unchanged findings
- crawl coverage deltas when the inputs come from ‘crawl‘
- a short terminal note when crawl coverage changes

Example:

```
.\venv\Scripts\python.exe -m app.main compare old.json new.json --html-output
```

‘doctor‘

‘doctor‘ checks the local machine and app environment, including suspicious listeners and outbound connections. It also adds a deployment profile hint so you can tell whether the host looks like a local development box, a service-backed server, or a likely VPS-style deployment. Its readiness score is paired with a short breakdown of the checks that most affected it, and the report labels the overall state as ‘ready‘, ‘warning‘, or ‘danger‘.

‘doctor‘ also accepts ‘--json-output‘, ‘--markdown-output‘, and ‘--html-output‘ if you want a saved report file.

What it checks:

- OS info
- Python version
- ‘.env‘ presence
- output folder writability
- app config paths

- common Nginx config paths
- localhost ports
- safe secret-status signals without printing secret values

Example:

```
.\venv\Scripts\python.exe -m app.main doctor --env-file C:\path\to\autoentrytrack\.env
```

'server-check'

'server-check' is the server-facing discovery flow.

What it does:

- checks Nginx and systemd hints
- discovers the app target
- resolves the apps own '.env' when possible
- scans the discovered local target
- surfaces the deployment profile hint and the overall 'ready' / 'warning' / 'danger' state

Important flags:

- '--env-file'
- '--nginx-config'
- '--timeout'
- '--yes'

Example:

```
.\venv\Scripts\python.exe -m app.main server-check --yes
```

'fix --local'

'fix --local' is the first real live-edit lane.

What it does:

1. discovers a supported local file
2. backs it up
3. applies the change
4. validates it

5. rolls back on failure

What to expect:

- the target file is backed up before any edit
- the terminal shows the backup path and validation command
- the chosen local fix must be supported by the current lane
- unsupported findings stay in generate-artifact mode

Example:

```
.\venv\Scripts\python.exe -m app.main fix --local --yes
```

'--generate-fixes' and the legacy '--apply-fixes'

'--generate-fixes' creates local remediation artifacts and notes.

'--apply-fixes' still works as a backwards-compatible alias, but '--generate-fixes' is the current name.

What to expect:

- a backup of the generated artifact, if one already exists
- a file under 'outputs/generated/'
- a matching note and audit entry

'demo'

'demo' runs a local test target for development.

Example:

```
.\venv\Scripts\python.exe -m app.main demo --port 8000
```

'demo-site' remains available as a compatibility alias.

'env' Variables

These are the main environment variables PsyberShield reads:

Variable	Used by	Purpose
---	---	---

```
| 'APP_URL' | 'scan' | Primary fallback target |
| 'TARGET_URL' | 'scan' | Secondary fallback target |
| 'BASE_URL' | 'scan' | Final fallback target |
| 'DEBUG' | 'doctor', 'server-check' | Warns about noisy debug mode |
| 'SECRET_KEY' | 'doctor', 'server-check' | Presence/weakness check only |
| 'SERVER_NAME' | 'doctor', 'server-check' | Reported as present or missing |
| 'DATABASE_URL' | 'doctor', 'server-check' | Reported as present or missing |
| 'SMTP_PASSWORD' | 'doctor', 'server-check' | Reported as present or missing |
| 'PsyberShield_PASSWORD' | 'scan', 'crawl' | Secret value read by '--password-env' from the s
```

Saved Outputs

Current output folders:

- 'outputs/' for user-facing artifacts
- 'outputs/generated/' for generated fix artifacts
- 'outputs/backups/' for backup files
- 'outputs/remediation/' for remediation notes

Useful output examples:

```
.\env\Scripts\python.exe -m app.main scan https://example.com --html-output
.\env\Scripts\python.exe -m app.main crawl https://example.com --markdown-output outputs\crawl
.\env\Scripts\python.exe -m app.main audit --json-output
```

What the Output Means

Typical scan output includes:

- target summary
- finding counts
- severity counts
- top categories
- WAF or CDN signals
- TLS summary when available
- a findings table
- proposed fixes for report-only issues

Typical crawl output adds:

- pages visited
- unique URLs

- seed sources
- per-page findings
- grouped repeated findings with affected URL lists

Typical doctor or server-check output adds:

- root path
- OS
- Python version
- environment file discovery
- Nginx/systemd hints
- suspicious listener and outbound connection activity
- localhost port checks

Typical incident output adds:

- source log paths
- suspect IPs and blocked IPs
- log family hints such as 'apache-access', 'apache-error', 'auth', 'auth-middleware', 'unicorn', 'ssh', 'sudo', 'systemd', and 'uwsgi'
- optional denylist, fail2ban, rate-limit, or maintenance-mode containment artifacts

Typical timeline output adds:

- ordered finding and containment events
- timestamps when the log lines or audit entries provided them
- source paths and audit log references for each event

Limitations and Boundaries

PsyberShield is intentionally defensive and best effort:

- it is not a substitute for a full SIEM, IDS, or dedicated EDR platform
- browser auth depends on the page structure, selectors, and whether the app accepts the login flow you provide
- some findings are heuristic and may need manual review before you treat them as confirmed issues
- WAF, TLS, and process/port signals may be unavailable or incomplete depending on the target and local environment
- saved reports and bundles only include what you asked PsyberShield to capture
- only the supported local fix lane is applied automatically; other findings stay in report or artifact form unless you act on them yourself

- you should only scan systems you own or have explicit permission to test

Integrity

'integrity' compares monitored files in a root directory against an optional saved baseline.

Example:

```
.\env\Scripts\python.exe -m app.main integrity . --baseline baselines\integrity.json --json-o
```

Typical integrity output adds:

- the monitored root
- the baseline path when one is supplied
- changed, missing, and new monitored files
- the file category, kind, hash, size, and timestamp for each tracked file

Baseline Workflow

Create a baseline after a known-good deployment:

```
.\env\Scripts\python.exe -m app.main integrity . --create-baseline baselines\integrity.json
```

Refresh the baseline after approved changes only:

```
.\env\Scripts\python.exe -m app.main integrity . --create-baseline baselines\integrity.json
```

Use the baseline in 'watch' when you want live drift checks:

```
.\env\Scripts\python.exe -m app.main watch --baseline baselines\integrity.json --logs outputs
```

Operational notes:

- keep the baseline file in a team-owned location
- refresh it only after approved changes land
- expect ordinary drift from releases, config edits, and content updates
- record what changed when you refresh it so the baseline stays trusted

Drift

'drift' compares two saved reports of the same type and surfaces changes that matter

for defensive operations.

Example:

```
.\venv\Scripts\python.exe -m app.main drift baselines\scan.json outputs\scan.json --html-output
```

Typical drift output adds:

- the report type being compared
- baseline and current report paths
- changed findings or checks
- file, log, header, or config drift summaries depending on the report type

Secret Exposure

'secrets' scans likely config and log files for obvious secret exposure and redacts the evidence before it is stored in the report.

Example:

```
.\venv\Scripts\python.exe -m app.main secrets . --json-output outputs\secrets.json
```

If you do not pass any output flags, 'secrets' writes JSON, Markdown, and HTML reports under 'outputs/' automatically.

Typical secret-exposure output adds:

- the monitored root
- candidate source file count
- redacted snippets from matching lines
- a short recommendation for each exposure

Vulnerability Inventory

'vuln scan' inventories local software versions and compares them with a small bundled offline advisory ruleset.

Example:

```
.\venv\Scripts\python.exe -m app.main vuln scan
.\venv\Scripts\python.exe -m app.main vuln scan --json-output outputs\vuln.json --html-output
.\venv\Scripts\python.exe -m app.main vuln scan --inventory-only
```

This version checks common local commands such as Nginx, Apache, OpenSSL, Python, Node, npm, and PHP, then records the versions it can prove. CVE matching currently uses bundled offline rules for a few well-known advisories, including Apache HTTP Server 2.4.49/2.4.50 and OpenSSL 3.0.0 through 3.0.6. It does not query NVD, OSV, distro advisories, or other live CVE feeds yet.

Typical vulnerability-inventory output adds:

- the monitored root
- the software components checked
- versions discovered from local command output
- local advisory matches when the bundled ruleset applies
- a note that distro backports and vendor advisories should be confirmed

Bundle

'bundle' packages a report together with related artifacts into a ZIP archive.

Example:

```
.\venv\Scripts\python.exe -m app.main bundle outputs\incident.json --artifact outputs\incident
```

If you omit '--bundle-output', PsyberShield creates a report-named archive like 'incident.bundle.zip' and writes a matching manifest inside the ZIP.

Typical bundle output adds:

- the archive path
- the source report path
- the files included in the archive
- a small manifest inside the ZIP

Notifications

'incident', 'integrity', and 'timeline' can send a short report summary after they finish.

Common flags:

- '--webhook-url'
- '--slack-webhook-url'

- '--discord-webhook-url'
- '--email-to'
- '--email-from'
- '--smtp-host'
- '--smtp-port'
- '--smtp-username'
- '--smtp-password-env'
- '--smtp-starttls' and '--no-smtp-starttls'

Example:

```
.\venv\Scripts\python.exe -m app.main incident --logs outputs\access.log --webhook-url https://
.\venv\Scripts\python.exe -m app.main integrity . --baseline baselines\integrity.json --email-
.\venv\Scripts\python.exe -m app.main timeline outputs\incident.json --audit-log outputs\audit
```

Troubleshooting

If a bare output flag creates a file and you cannot find it:

- PsyberShield writes under 'outputs/' in the current project directory
- if you pass '\outputs\file.html', PsyberShield treats it as project-relative and tells you the resolved path
- if you pass a full absolute path, PsyberShield respects it

If 'crawl' only reaches login or redirect pages:

- add auth flags
- add '--auth-check-url'
- confirm that the login worked before you trust the crawl results

If 'server-check' finds a local app target but still points to the repo '.env':

- make sure the discovered 'EnvironmentFile' or working directory is correct
- pass '--env-file' if you want to override discovery

Maintaining This Guide

This guide is meant to be updated as PsyberShield grows.

When you add a new command or flag:

1. update the CLI help in 'app/main.py'

2. update this guide
3. update the README quick-start sections if needed
4. regenerate the PDF

Architecture

Source: `'docs/architecture.md'`

PsyberShield Architecture

Purpose

PsyberShield is a Python-based security visibility and response tool for small servers and web applications.

It supports HTTP/HTTPS scanning, passive crawling, detection of common website/server misconfigurations, WAF/firewall impact notes, incident log analysis for Apache/auth/systemd-style logs, continuous watch monitoring for logs/files/process activity, file integrity monitoring, baseline drift detection, secret exposure checks, report bundling, report notifications, and safe defensive fixes where possible.

The preferred CLI command is `'pshield'`. `'psybershield'` and `'turan'` remain compatibility aliases during the transition.

For a command-by-command usage guide, see `[docs/turan-user-guide.md](docs/turan-user-guide.md)` and the matching PDF at `[psybershield-user-guide.pdf](psybershield-user-guide.pdf)`.
For a short version history, see `[docs/changelog.md](docs/changelog.md)`.

The core flow is:

1. Observe
2. Classify
3. Recommend
4. Apply only safe reversible fixes

Anything that changes the system must pass through one gate:

```
if finding.fix_level > allowed_fix_level:  
    return "approval_required"
```

That gate lives in one place only: the remediation executor.

Safety Principles

- Report-only findings stay read-only.
- Safe fixes must be reversible.
- Every fix plan must include rollback metadata.
- Approval is required for anything outside the allowed fix level.
- Secrets must never be printed into reports or logs.
- The executor is the only module that can perform changes.
- All other modules produce findings, plans, or recommendations only.
- The first live-edit lane is 'fix --local', and it only touches a discovered server file after backup and validation.
- The incident-response lane is 'incident', and it only writes a denylist include after analysis and confirmation.
- The watch lane is 'watch', and it stays log-only in v1: it observes, scores, and recommends containment without auto-blocking.

Working Agreement

These rules govern how we build PsyberShield together:

1. Comments should be short working notes, not tutorial prose.
2. If a comment is needed, keep it close to the code and make it practical.
3. Use comments for the lines that need a nudge, not for everything.
4. Prefer code that reads clearly without a lot of explanation.
5. Function names should read naturally to a human.
6. If a function name is unclear, ask before creating it.
7. When asking about a function name, explain its purpose first so the naming choice is easy to review.
8. Every file we create should be actively used and connected to the rest of the app.
9. Do not create placeholder functions that live in isolation or force a file to exist without purpose.
10. Prefer small, related files that each earn their place in the project.

Comment style examples:

- Good: '# normalize input'
- Good: '# check v1 headers'
- Bad: '# We normalize the input first so the rest of the scan can trust one canonical URL.'
- Bad: '# We focus on a short list of headers that are easy to verify and easy to explain to users.'

See also: docs/working-agreement.md

Cleaned Project Tree

```
PsyberShield/
|-- app/
|   |-- __init__.py
|   |-- main.py           # CLI entry point
|   |-- config.py         # scan settings, defaults, environment handling
|   |-- artifacts.py      # load saved scan reports
|   |-- comparison.py     # compare two saved scan reports
|   |-- doctor.py         # local machine and environment checks
|   |-- incident.py       # suspicious activity analysis and containment planning
|   |-- watch.py          # continuous monitoring and risk scoring
|   |-- scanner.py        # coordinates scan flow
|   |-- models.py         # Target, Finding, FixPlan, ScanResult, ComparisonResult
|   |-- policy.py         # fix levels, approval rules, redaction rules
|   |-- approvals.py      # admin approval flow for Level 2 actions
|   |-- audit.py          # append-only audit trail for scans and fixes
|   |-- redaction.py      # secret masking for logs and reports
|   |
|   |-- http/
|   |   |-- __init__.py
|   |   |-- client.py     # safe HTTP client with timeouts and limits
|   |   |-- crawler.py    # passive crawler, same-scope only
|   |   |-- normalizer.py # URL cleaning, canonicalization, scope checks
|   |
|   |-- checks/
|   |   |-- __init__.py
|   |   |-- headers.py    # security headers
|   |   |-- cookies.py    # Secure, HttpOnly, SameSite
|   |   |-- tls.py        # HTTPS/cert summary
|   |   |-- exposed_files.py # .env, .git, backup files, index exposure
|   |   |-- server_info.py # server banners and version disclosure
|   |   |-- waf.py        # WAF/CDN detection note and scan-confidence hints
|   |
|   |-- hardening/
|   |   |-- __init__.py
|   |   |-- recommendations.py # fix suggestions and remediation plans
|   |   |-- executor.py       # applies only approved, reversible fixes
|   |   |-- backup.py         # creates restore points before changes
|   |   |-- incident.py       # denylist containment for suspicious activity
```

```

| | '-- local_fixes.py      # first real local edit lane for a discovered server file
| | '-- nginx.py           # safe nginx snippet templates only
| |
| '-- reports/
|     |-- __init__.py
|     |-- console.py       # terminal output
|     |-- incident_report.py # incident report export
|     |-- watch_report.py   # watch report export
|     |-- json_report.py    # JSON export
|     |-- markdown_report.py # Markdown export
|     |-- html_report.py    # optional later
|
|-- tests/
|     |-- test_policy.py
|     |-- test_models.py
|     |-- test_executor_gate.py
|     '-- test_redaction.py
|-- pyproject.toml
|-- README.md
|-- .env.example
'-- docs/
    '-- architecture.md

```

Data Models

Use Pydantic models for all structured scan and remediation data.

Target

```
from pydantic import BaseModel, Field, HttpUrl
```

```

class Target(BaseModel):
    url: HttpUrl
    scheme: str = Field(pattern=r"^https?$")
    host: str
    port: int | None = None
    scope_root: HttpUrl | None = None

```

Finding

```
from pydantic import BaseModel, Field
```

```

class Finding(BaseModel):
    id: str
    target_url: str
    title: str
    description: str
    severity: str = Field(pattern=r"^(info|low|medium|high|critical)$")
    category: str
    evidence: dict[str, str | int | bool | None]
    fix_level: int = Field(ge=0, le=3)
    risk_level: str = Field(pattern=r"^(low|medium|high|critical)$")
    requires_approval: bool = False
    backup_path: str | None = None
    rollback_command: str | None = None
    expected_impact: str = ""
    references: list[str] = Field(default_factory=list)

```

FixPlan

```

from pydantic import BaseModel, Field

```

```

class FixPlan(BaseModel):
    finding_id: str
    fix_level: int = Field(ge=0, le=3)
    risk_level: str = Field(pattern=r"^(low|medium|high|critical)$")
    requires_approval: bool = False
    backup_path: str | None = None
    rollback_command: str | None = None
    expected_impact: str
    status: str = Field(default="proposed", pattern=r"^(proposed|approved|applied|blocked)$")
    approved_by: str | None = None
    approved_at: str | None = None
    applied_at: str | None = None

```

Policy

```

from pydantic import BaseModel, Field

```

```

class Policy(BaseModel):

```

```
allowed_fix_level: int = Field(default=0, ge=0, le=3)
require_backup_for_level_1: bool = True
require_approval_for_level_2: bool = True
block_level_3: bool = True
redact_secrets_in_reports: bool = True
redact_secrets_in_logs: bool = True
audit_log_path: str = "outputs/audit.log"
max_crawl_depth: int = Field(default=2, ge=0, le=10)
max_pages: int = Field(default=100, ge=1, le=10_000)
timeout_seconds: float = Field(default=10.0, gt=0)
```

ScanResult

```
from pydantic import BaseModel
```

```
class ScanResult(BaseModel):
    target: Target
    findings: list[Finding]
    fix_plans: list[FixPlan] = Field(default_factory=list)
    scanned_urls: list[str] = Field(default_factory=list)
    notes: list[str] = Field(default_factory=list)
    waf_signals: list[str] = Field(default_factory=list)
    scan_confidence: float = 1.0
```

ComparisonResult

```
from pydantic import BaseModel, Field
```

```
class ComparisonFinding(BaseModel):
    finding_id: str
    title: str
    category: str
    severity: str = Field(pattern=r"^(info|low|medium|high|critical)$")
    change: str = Field(pattern=r"^(fixed|new|unchanged)$")
```

```
class ComparisonResult(BaseModel):
    old_report: str
    new_report: str
    fixed_findings: list[ComparisonFinding] = Field(default_factory=list)
```

```
new_findings: list[ComparisonFinding] = Field(default_factory=list)
unchanged_findings: list[ComparisonFinding] = Field(default_factory=list)
old_risk_score: int = 0
new_risk_score: int = 0
risk_trend: str = Field(pattern=r"^(improved|worsened|unchanged)$")
```

Fix Levels

Level 0: Report only

- Missing headers
- Open ports or service exposure observations
- Weak cookie flags
- Outdated packages

Level 1: Safe auto-fix

- Create report files
- Tighten app-owned file permissions
- Add non-breaking comments or config suggestions
- Rotate scanner logs
- Create backups before changes

Level 2: Approval required

- Change Nginx configs
- Restart services
- Modify firewall rules
- Change systemd files
- Update packages
- Edit app environment variables

Level 3: Never auto-do

- Delete files
- Disable SSH
- Kill unknown processes
- Change database contents
- Expose secrets in reports

Executor Gate

The remediation executor is the only module that can apply a fix.
It must enforce policy before any write, restart, or system change.

```
from __future__ import annotations

from app.models import Finding, FixPlan, Policy


def execute_fix(finding: Finding, plan: FixPlan, policy: Policy) -> str:
    # Single gate.
    if finding.fix_level > policy.allowed_fix_level:
        return "approval_required"

    if plan.fix_level > policy.allowed_fix_level:
        return "approval_required"

    if plan.fix_level >= 2 and policy.require_approval_for_level_2:
        return "approval_required"

    if plan.fix_level >= 3 and policy.block_level_3:
        return "blocked"

    if policy.require_backup_for_level_1 and plan.fix_level >= 1 and not plan.backup_path:
        return "blocked"

    if not plan.rollback_command:
        return "blocked"

    if plan.expected_impact.strip() == "":
        return "blocked"

    # Safe, reversible changes only.
    return "applied"
```

Approval Flow

Level 2 actions must be explicit:

1. A finding produces a FixPlan.
2. Policy marks the plan as requiring approval.
3. The approver reviews backup_path, rollback_command, and expected_impact.
4. The executor applies the change only after approval is recorded.

5. The audit log stores the decision and result.

Audit Requirements

Every scan and fix action should write an audit event with:

- timestamp
- target
- action
- actor
- policy level
- approval status
- result
- rollback metadata

Audit records should be append-only.

Default path:

- 'outputs/audit.log'

Redaction Rules

Reports and logs must redact:

- tokens
- secrets
- passwords
- private keys
- session cookies
- authorization headers

Never emit raw secrets into HTML, Markdown, JSON, or console output.

Recommended Build Order

1. 'models.py'
2. 'policy.py'
3. 'redaction.py'
4. 'http/client.py'
5. 'http/normalizer.py'
6. 'checks/headers.py'

7. 'checks/cookies.py'
8. 'checks/tls.py'
9. 'hardening/recommendations.py'
10. 'hardening/executor.py'
11. 'reports/json_report.py'
12. 'reports/markdown_report.py'
13. 'tests/test_policy.py'
14. 'tests/test_executor_gate.py'

Notes for v1

- Keep crawling passive and same-scope only.
- Do not bundle active exploitation features.
- Do not auto-edit system files or services without approval.
- Prefer small, reversible changes that are easy to audit.

CLI Shape

Use 'scan' for one-page inspection and 'crawl' for multi-page in-scope walking:

```
.\env\Scripts\python.exe -m app.main scan https://example.com
.\env\Scripts\python.exe -m app.main scan https://example.com --yes
.\env\Scripts\python.exe -m app.main crawl https://example.com --max-pages 20 --max-depth 2
.\env\Scripts\python.exe -m app.main crawl https://example.com --include /auth/ --exclude /lo
.\env\Scripts\python.exe -m app.main crawl https://example.com --allow-offsite
.\env\Scripts\python.exe -m app.main crawl https://example.com --seed-robots --seed-sitemap
.\env\Scripts\python.exe -m app.main crawl https://example.com --yes
```

Authenticated crawl roadmap:

- Phase 1, implemented: HTTP login/session reuse, JSON or form payloads, custom field names, raw cookie reuse, and an optional protected-page check after login.
- Phase 2, implemented: saved session import/export with '--session-file' and '--save-session'.
- Phase 3, implemented: browser storage-state import/export with '--storage-state' and '--save-storage-state'.
- Phase 4, implemented: browser automation support for JS-heavy login flows when HTTP/session reuse is not enough. Install the optional browser extra with 'pip install [browser]'. Browser auth requires '--login-url' or '--storage-state' so the session can be established first.

Example phase-1 commands:

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --login-url /auth/login --auth
.\env\Scripts\python.exe -m app.main crawl https://example.com --cookie "session=abc123; csrf"
```

Example phase-2 command:

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --session-file sessions\autoen
```

Example phase-3 commands:

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --storage-state browser\storag
```

To reuse an existing browser session without refreshing it, point '--storage-state' at the saved file and omit '--save-storage-state'.

Example phase-4 command:

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --auth-method browser --login-
```

Improvement Roadmap

The planned phases are complete. We are keeping the app stable while doing final release prep and cleanup.

Phase 1: onboarding and packaging

- clearer 'pshield --help' output
- 'pshield demo' and 'pshield doctor' first-run guidance
- packaged '.exe' build path and release notes

Phase 2: crawl and auth UX

- better page discovery explanations
- smarter auth/session handling
- clearer 'why only these pages?' messaging
- polished sitemap and robots support

Phase 3: reports

- cleaner HTML report
- executive summary
- severity explanation

- 'what to fix first'

Phase 4: doctor and server-check

- better VPS detection
- Nginx, systemd, and env checks
- clear 'ready', 'warning', and 'danger' status

Phase 5: profiles

- '--profile quick' for a fast, shallow pass
- '--profile full' for a broader crawl with robots and sitemap hints
- '--profile safe-vps' for a balanced VPS-friendly pass

Example:

```
.\venv\Scripts\python.exe -m app.main crawl https://example.com --profile safe-vps
.\venv\Scripts\python.exe -m app.main scan https://example.com --profile quick
```

Phase 6: fix confidence

- 'report only'
- 'generate artifact'
- 'safe local fix'
- 'needs manual approval'

Release prep:

- verify the packaged '.exe'
- refresh the user guide PDF if the docs change again
- run a final focused test pass before cutting a release

Defensive Engine Roadmap

The next major evolution is split into advisory intelligence and containment planning. The guiding rule is that destructive actions must be explicit, reversible where possible, and audited.

Phase 1: advisory abstraction

- add 'AdvisorySource'
- add 'AdvisoryQuery'

- add 'LocalRulesSource'
- route the existing bundled offline CVE rules through the source abstraction
- keep 'vuln scan' behavior compatible

Phase 2: OSV dependency advisories

- support Python dependency manifests first
- parse 'requirements.txt'
- parse 'pyproject.toml' where feasible
- add local cache and graceful offline handling
- label findings as confirmed, potential, or unknown

Phase 3: version and distro awareness

- normalize package names and versions
- separate upstream version matches from distro-patched package status
- report vendor backport uncertainty clearly

Phase 4: containment planning

- add 'ContainmentRecommendation'
- add 'ContainmentAction'
- add 'ContainmentResult'
- make 'watch' produce recommendations before any action execution exists

Phase 5: containment artifacts

- generate Windows Firewall, iptables/nftables, Nginx denylist, and fail2ban artifacts
- keep artifact generation separate from application

Phase 6: dry-run containment

- add 'watch --contain --dry-run'
- audit the plan without changing the system

Phase 7: low-risk temporary blocking

- add explicit temporary IP containment behind policy approval
- audit every applied action
- include rollback guidance

Phase 8: high-risk actions

- consider process kill, account disablement, and file quarantine only behind strict policy
- require explicit flags, confidence thresholds, audit records, and rollback or restoration guidance

Optional report exports:

```
.\env\Scripts\python.exe -m app.main scan https://example.com --json-output outputs\scan.json
```

```
.\env\Scripts\python.exe -m app.main scan https://example.com --html-output outputs\scan.html
```

```
.\env\Scripts\python.exe -m app.main scan https://example.com --html-output
```

If you pass one of the report output flags without a path, PsyberShield creates a timestamped file under 'outputs/'.

When discovery resolves a local app target, the saved JSON, Markdown, and HTML reports include an 'Application Context' section with the resolved target, env source, server hints, and discovery notes.

When crawl mode visits multiple pages, the saved JSON, Markdown, and HTML reports also include a 'Scanned URLs' section with the in-scope pages PsyberShield visited, and repeated findings are grouped with an 'Affected URLs' list instead of being repeated on every page.

If you pass a Windows-rooted output path like '\outputs\scan.html', PsyberShield treats it as project-relative and prints the resolved path instead of silently writing to the drive root.

Cookie findings stay cautious about CSRF-style values: when the cookie name looks like a CSRF/XSRF token, PsyberShield lowers confidence and recommends reviewing the issuance location before making a flag change.

Saved report rendering:

```
.\env\Scripts\python.exe -m app.main report outputs\scan.json --html-output outputs\scan.html
```

'report' accepts '.json', '.md', and '.html' files.

```
.\env\Scripts\python.exe -m app.main report outputs\scan.md
```

```
.\env\Scripts\python.exe -m app.main report outputs\scan.html
```

Audit history:

```
.\env\Scripts\python.exe -m app.main audit --last 25
.\env\Scripts\python.exe -m app.main audit --event scan
.\env\Scripts\python.exe -m app.main audit --target example.com
.\env\Scripts\python.exe -m app.main audit --audit-log outputs\audit.log
.\env\Scripts\python.exe -m app.main audit --json-output outputs\audit.json
```

Doctor:

```
.\env\Scripts\python.exe -m app.main doctor
```

'doctor' checks the local machine, config paths, open localhost ports, safe environment status, a deployment profile hint, and any resolved app target without taking a target URL. The readiness score includes a short breakdown of the checks that most affected it, and the report also labels the overall state as 'ready', 'warning', or 'danger'.

'doctor' also supports '--json-output', '--markdown-output', and '--html-output' for saved reports.

'doctor' can also read a specific env file:

```
.\env\Scripts\python.exe -m app.main doctor --env-file /path/to/autoentrytrack/.env
```

Server check:

```
.\env\Scripts\python.exe -m app.main server-check
```

'server-check' stays focused on server-facing paths, local service signals, and config checks, then scans the resolved local target when one is found. It also prints the deployment profile hint and the overall readiness state so the host state is easier to read at a glance.

'server-check' can also read a specific env file and a specific Nginx config:

```
.\env\Scripts\python.exe -m app.main server-check --env-file /path/to/autoentrytrack/.env --nginx-config /path/to/nginx.conf
```

Incident response:

```
.\env\Scripts\python.exe -m app.main incident --logs /var/log/nginx/access.log --apply-blocks
```

'incident' analyzes suspicious activity in log files, groups repeated attacker

signals, and can write denylist, fail2ban, rate-limit, or maintenance-mode containment presets when you explicitly opt into containment.

'timeline' turns a saved incident report and optional audit log into a chronological view of log-derived findings and containment actions.

Scan fallback:

```
.\venv\Scripts\python.exe -m app.main scan
```

When 'scan' runs without a URL, PsyberShield looks for 'APP_URL', then 'TARGET_URL', then 'BASE_URL' in '.env' or the current environment. If those are missing, it checks the server layout first and prefers the app's own '.env' when Nginx or systemd point to an app root or an explicit 'EnvironmentFile'. If discovery still can't resolve a target, PsyberShield falls back to the project '.env' only as the last local fallback.

When discovery finds a local target, PsyberShield prints a short 'Discovery:' line first and then the fuller context block.

'scan' can also read a specific env file:

```
.\venv\Scripts\python.exe -m app.main scan --env-file /path/to/autoentrytrack/.env
```

'.env' variables:

Variable	Used by	Meaning
---	---	---
'APP_URL'	'scan'	Default target URL when you skip the argument
'TARGET_URL'	'scan'	Backup target URL if 'APP_URL' is missing
'BASE_URL'	'scan'	Final fallback target URL
'DEBUG'	'doctor', 'server-check'	Flags a noisy local debug setup
'SECRET_KEY'	'doctor', 'server-check'	Checked for presence and weak values only
'SERVER_NAME'	'doctor', 'server-check'	Reported as present or missing
'DATABASE_URL'	'doctor', 'server-check'	Reported as present or missing
'SMTP_PASSWORD'	'doctor', 'server-check'	Reported as present or missing

Baseline snapshots:

```
.\venv\Scripts\python.exe -m app.main baseline https://example.com --output baselines\example.
```

Friendly baseline names:

```
.\env\Scripts\python.exe -m app.main baseline https://example.com --label vps-west
```

Each baseline snapshot writes a small companion metadata file next to it, like 'baselines\vps-west.json.meta.json', so the resolved target and discovery trail stay attached to the saved baseline.

Audit log override:

```
.\env\Scripts\python.exe -m app.main scan https://example.com --audit-log outputs\audit.log
.\env\Scripts\python.exe -m app.main baseline https://example.com --audit-log outputs\audit.log
```

Compare runs:

```
.\env\Scripts\python.exe -m app.main compare outputs\old.json outputs\new.json
```

When the saved reports came from 'crawl', compare also shows crawl coverage deltas, including added and removed pages.

When crawl coverage changes, compare prints a short terminal note with the added and removed page counts.

Comparison reports:

```
.\env\Scripts\python.exe -m app.main compare old.json new.json --markdown-output compare.md
.\env\Scripts\python.exe -m app.main compare old.json new.json --html-output compare.html
```

Local demo site:

```
.\env\Scripts\python.exe -m app.main demo --port 8000
.\env\Scripts\python.exe -m app.main scan http://127.0.0.1:8000
```

Timeout control:

```
.\env\Scripts\python.exe -m app.main scan https://example.com --timeout 5
```

Policy file:

```
.\env\Scripts\python.exe -m app.main scan https://example.com --policy policy.json
```

Start from policy.example.json and edit the values you need.

Preview fixes:

```
.\venv\Scripts\python.exe -m app.main scan http://127.0.0.1:8000 --preview-fixes
```

Interactive fixes:

```
.\venv\Scripts\python.exe -m app.main scan http://127.0.0.1:8000 --interactive
```

Interactive mode shows a numbered fix list, labels each one as report only, generate artifact, safe local fix, or needs manual approval, asks whether you want to generate artifacts or apply fixes locally, and then lets you choose all fixes or a numbered subset.

If there are more than ten fixes, PsyberShield shows a paged list and lets you move with 'n' and 'p'.

For 'fix-local', PsyberShield shows the target file, backup path, validation command, and rollback state before the final apply confirmation.

Apply fixes:

```
.\venv\Scripts\python.exe -m app.main scan http://127.0.0.1:8000 --generate-fixes
```

Apply mode creates the backup first, then writes the local remediation note for each allowed safe fix.

'--apply-fixes' is kept as a legacy alias for '--generate-fixes' so older muscle memory still works.

Each remediation note keeps the backup path in the footer when PsyberShield creates one.

The generated fix artifact itself lands under 'outputs/generated/' so it stays app-owned and easy to review.

Real local edit lane:

```
.\venv\Scripts\python.exe -m app.main fix --local
```

'fix --local' is the first live-edit path. It discovers a supported server file, backs up the real file first, applies one small reversible edit, validates the config, and rolls back on failure.

Changelog

Source: 'docs/changelog.md'

PsyberShield Changelog

This file tracks user-facing changes to PsyberShield over time.
It is intentionally short and practical so it can stay current as the app grows.

Unreleased

- Added the living user guide in Markdown and PDF form.
- Added optional output flags that can auto-create files under 'outputs/' when no path is supplied.
- Added output-path normalization for Windows-rooted relative paths like '\outputs\scan.html'.
- Added 'crawl' scope controls, seed sources, and grouped repeated findings.
- Added generic authenticated scan and crawl support, including browser-assisted login flows.
- Added 'fix --local' as the first real live-edit lane with backup, validation, and rollback.
- Added 'doctor' and 'server-check' discovery workflows for local and VPS environments, including a deployment profile hint and explicit 'ready', 'warning', and 'danger' state labels.
- Added '--profile quick', '--profile full', and '--profile safe-vps' presets for scan and crawl defaults.
- Added fix confidence labels for suggested actions: report only, generate artifact, safe local fix, and needs manual approval.
- Added 'incident' log analysis with Apache/auth/systemd patterns, optional Nginx denylist containment, and fail2ban-style output for suspicious activity.
- Added live log snapshot ingestion for 'incident' from 'journalctl', Windows Event Log, and file tails.
- Added richer incident signatures for Apache, Nginx, Unicorn, uWSGI, SSH, sudo, and auth middleware.
- Added 'integrity' file monitoring with baseline comparison for config files, web roots, and startup scripts.
- Added process and port checks that flag suspicious listeners and outbound connections in 'doctor' and 'server-check'.
- Added 'watch' as a hybrid monitoring command for log, file drift, and process activity with snapshot and follow modes.
- Added a readiness breakdown under the 'doctor' readiness score so the score explains the main warnings.
- Added optional JSON, Markdown, and HTML saved reports for 'doctor'.
- Added rate-limit and maintenance-mode containment presets alongside the Nginx denylist and fail2ban artifacts.
- Added 'timeline' reports that order findings, containment actions, and audit events chronologically.
- Added report notification hooks for webhooks, Slack, Discord, and email across

- `'incident'`, `'integrity'`, and `'timeline'`.
- Added a short `'compare'` terminal note when crawl coverage changes between saved crawl reports.
- Added `'drift'` for baseline drift detection across saved reports.
- Added `'secrets'` for redacted secret exposure checks in logs and config files, with default saved report outputs when none are supplied.
- Added `'vuln scan'` for local software inventory with bundled offline advisory matching for a small initial CVE ruleset.
- Added an advisory source abstraction and routed the bundled local CVE rules through `'LocalRulesSource'`.
- Added `'bundle'` for packaging reports and related containment artifacts into ZIP archives.

Notes

- When you add a new command, flag, or output format, update:
 - `'README.md'`
 - `'docs/architecture.md'`
 - `'docs/turan-user-guide.md'`
 - this changelog
- If the PDF guide changes, regenerate `'psybershield-user-guide.pdf'` with `'python generate_user_guide_pdf.py'`

Release Checklist

Before you call a version ready, check:

1. The CLI help reflects the current commands and flags.
2. The README and architecture doc mention the new behavior.
3. The user guide is updated for any new user-facing flow.
4. The PDF guide is regenerated from `'docs/turan-user-guide.md'`.
5. The Desktop app copy is synced.
6. The full test suite passes in the Desktop app copy.
7. Any new output paths still land under `'outputs/'` unless the user explicitly provides a full path.